

Parallelizzazione con MPI del solutore Navier–Stokes–Brinkman

Relazione sul lavoro svolto

1. Di cosa si tratta

Il programma risolve le equazioni di Navier–Stokes–Brinkman su una griglia tridimensionale, con lo schema a separazione delle direzioni visto a lezione. Finora girava su un solo processore.

L’obiettivo è farlo girare su più processori contemporaneamente usando MPI, la libreria standard del calcolo parallelo. L’idea di fondo è semplice: dividere la griglia in blocchi, assegnarne uno a ciascun processore, e far sì che quando a un processore serve un dato che sta a casa del vicino, glielo chieda con un messaggio.

Il lavoro è diviso in sette fasi, dalla A alla G. Le prime due preparano il terreno senza cambiare il modo in cui il solutore calcola; le successive collegano i pezzi, e alla fine il programma gira davvero su più processori e si misura quanto ci si guadagna.

In questa relazione ogni pezzo è mostrato con il codice vero e spiegato subito dopo.

2. Perché non basta “accendere” MPI

Ci sono due ostacoli, di natura molto diversa.

Primo ostacolo: il codice credeva di avere sempre tutta la griglia

Le dimensioni della griglia erano tre costanti fissate al momento della compilazione: `WIDTH`, `HEIGHT`, `DEPTH`. Ogni ciclo, ogni calcolo di posizione e ogni controllo del tipo “sono sul bordo?” le usava direttamente.

Dividendo la griglia, ciascun processore ne possiede solo una fetta. Il codice non aveva alcun modo di distinguere *la griglia intera* da *la mia parte*: per lui erano la stessa cosa.

Secondo ostacolo: l’algoritmo di Thomas è una catena

Il solutore risolve sistemi tridiagonali con l’algoritmo di Thomas. Il valore in un punto si calcola da quello del punto precedente, che si calcola dal precedente ancora: è una catena rigidamente sequenziale.

Se spezziamo una linea fra due processori, il secondo non può cominciare finché il primo non ha finito. Non avremmo parallelizzato niente: stesso tempo di prima, più il costo dei messaggi. La soluzione, vista a lezione, è il *complemento di Schur*.

3. Fase A — La griglia diventa “un pezzo”

La struttura che sostituisce le costanti

Prima: Tre costanti di compilazione, usate ovunque.

Dopo: Una struttura dati, passata come parametro a ogni funzione che percorre la griglia.

```
typedef struct Decomp {
    int n_global[3]; /* dimensione della griglia intera */
    int n[3]; /* celle che possiedo io */
    int start[3]; /* indice globale della mia prima cella */
    size_t stride[3]; /* distanza in memoria fra celle vicine */
    size_t base; /* offset della prima cella posseduta */
    int is_first[3]; /* tocco la parete inferiore? */
}
```

```

    int is_last[3]; /* tocco la parete superiore? */
    size_t n_cells; /* celle da allocare per un campo */
} Decomp;

```

Con una griglia $32 \times 16 \times 24$ su un solo processore i campi valgono: `n_global` e `n` entrambi $\{32, 16, 24\}$, `start` $\{0, 0, 0\}$, `stride` $\{1, 32, 512\}$, `base` 0, tutti i flag a 1, e `n_cells` = 12 288.

Il campo che porta tutto il peso è `stride`. Significa: per spostarsi di una cella lungo *Y* bisogna avanzare di 32 posizioni nell’array, e per cambiare piano di 512 (cioè 32×16). Averlo memorizzato come *dato* invece di ricostruirlo ogni volta dalle dimensioni è ciò che renderà indolore la fase C.

Come si raggiunge una cella

```

static inline size_t decomp_index(const Decomp *d, int i, int j, int k) {
    return d->base + (size_t)i * d->stride[0] + (size_t)j * d->stride[1] +
           (size_t)k * d->stride[2];
}

static inline int decomp_global(const Decomp *d, int i, int component) {
    return d->start[component] + i;
}

```

La prima funzione traduce una posizione nella griglia in una posizione in memoria. Con i valori dell’esempio dà $0 + i + 32*j + 512*k$, cioè esattamente il calcolo che il codice faceva prima a mano: stessa aritmetica, ma con i numeri presi dalla struttura.

La seconda traduce un indice *locale* (“la mia terza cella”) in un indice *globale* (“la cella numero 15 del dominio”). Serve ogni volta che si calcola una coordinata fisica o si chiede se siamo su una parete.

Come si riempie la struttura

```

static void decomp_set_layout(Decomp *d, int halo) {
    size_t allocated_x = (size_t)d->n[0] + 2 * (size_t)halo;
    size_t allocated_y = (size_t)d->n[1] + 2 * (size_t)halo;
    size_t allocated_z = (size_t)d->n[2] + 2 * (size_t)halo;

    d->stride[0] = 1;
    d->stride[1] = allocated_x;
    d->stride[2] = allocated_x * allocated_y;
    d->base = (size_t)halo * (d->stride[0] + d->stride[1] + d->stride[2]);
    d->n_cells = allocated_x * allocated_y * allocated_z;
}

```

Questa è l’unica funzione che decide dove stanno fisicamente i dati. Il parametro `halo` è il margine di celle lasciato su ogni faccia: oggi vale zero, in fase C diventerà uno.

Il punto importante: **aggiungere il margine cambia solo questa funzione**. Con `halo` = 1 gli stride diventano $\{1, 34, 612\}$ e `base` diventa 647 invece di 0 – e le centinaia di chiamate a `decomp_index` sparse nel codice continuano a funzionare senza essere toccate.

I quattro tipi di modifica

Ogni singola modifica della fase A rientra in uno di questi quattro casi. I primi tre compaiono tutti insieme in questo ciclo, che riempie un campo di velocità.

Prima

```

size_t off = 0;
for (int k = 0; k < DEPTH; k++) {
    for (int j = 0; j < HEIGHT; j++) {
        for (int i = 0; i < WIDTH; i++) {

```

```

    Real x = centered_physical_coord(i, 0);
    ...
    v_x[off] = vector_fn(staggered_physical_coord(i, 0), y, z, time, 0);
    off++;
  }
}
}

```

Dopo

```

for (int k = 0; k < d->n[2]; k++) {
  int gk = decomp_global(d, k, 2);
  for (int j = 0; j < d->n[1]; j++) {
    int gj = decomp_global(d, j, 1);
    size_t row = decomp_index(d, 0, j, k);
    for (int i = 0; i < d->n[0]; i++) {
      int gi = decomp_global(d, i, 0);
      Real x = centered_physical_coord(gi, 0);
      ...
      size_t off = row + (size_t)i;
      v_x[off] = vector_fn(staggered_physical_coord(gi, 0), y, z, time, 0);
    }
  }
}
}

```

Tre cose sono cambiate.

- **Gli estremi dei cicli.** Da “scorro tutta la griglia” a “scorro le celle che possiedo”.
- **La posizione in memoria.** Il contatore `off++` avanzava di una posizione alla volta: funziona solo se le celle sono perfettamente consecutive. Con il margine non lo saranno più – dopo l’ultima cella di una riga vengono le celle di contorno, poi la riga successiva – e `off++` finirebbe dentro il margine. Ora la posizione si calcola.
- **La coordinata fisica.** Da `i` a `decomp_global(d, i, 0)`. La posizione di una cella nel mondo fisico dipende da dove sta nel *dominio*, non da dove sta nel mio pezzo di memoria. Senza questa conversione, il secondo processore calcolerebbe le forze e le condizioni al contorno come se si trovasse all’origine del dominio.

Il quarto caso è il più insidioso, e riguarda i controlli sul bordo.

Prima

```

if (i < 1 || i >= WIDTH - 1 || j < 1 || j >= HEIGHT || ...) {
  return 0.0;
}

```

Dopo

```

int gi = decomp_global(d, i, 0);
int gj = decomp_global(d, j, 1);
if (gi < 1 || gi >= d->n_global[0] - 1 || gj < 1 || ...) {
  return 0.0;
}

```

Prima, “ultima cella del mio array” e “sono sulla parete fisica” erano la stessa condizione. Con più processori le due cose si separano: un processore in mezzo al dominio ha una sua ultima cella, ma lì non c’è nessuna parete, c’è il vicino.

Sbagliare questo controllo crea *pareti fantasma* dentro il fluido: la simulazione gira, i numeri sembrano plausibili, e sono sbagliati. È il tipo di errore che si scopre solo confrontando i risultati fra numeri di processori diversi.

4. Fase B — I mattoni del parallelo

La fase B è divisa in quattro passi. Due dei quattro non usano MPI affatto: è voluto, e serve a separare la domanda “la matematica è giusta?” dalla domanda “la comunicazione è giusta?”. Alla fine della fase B tutti i pezzi esistono e sono verificati singolarmente, ma il solutore non è ancora parallelo: nessuno li ha ancora collegati.

B0 — Un muro fra MPI e il solutore

Prima: Nessun supporto per MPI.

Dopo: Due file nuovi che sono l’unico posto del progetto a conoscere MPI.

```
/* parallel.h */
void par_init(int *argc, char ***argv);
void par_finalize(void);
int par_rank(void); /* chi sono io: da 0 a par_size()-1 */
int par_size(void); /* quanti siamo */

/* parallel.c */
#ifdef USE_MPI
#include <mpi.h>
int par_rank(void) {
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    return rank;
}
#else
int par_rank(void) { return 0; } /* siamo un processo solo */
#endif
```

Il solutore chiama `par_rank()` e non sa che sotto ci sia `MPI_Comm_rank`. Non lo sa e non deve saperlo.

Il vantaggio non è estetico. Senza l’opzione di compilazione, le funzioni diventano banali e il programma seriale continua a funzionare come prima, senza MPI installato. La versione seriale resta così un **termine di paragone**: ogni volta che il parallelo darà un numero sospetto, lo confronteremo con la versione che sappiamo essere giusta.

B1 — Spezzare la catena, senza ancora usare MPI

Prima il metodo sequenziale, che resta il riferimento.

```
void thomas_solve(int n, const Real *a, const Real *b, const Real *c,
                  const Real *f, Real *x, Real *scratch) {
    Real denominator = b[0];
    scratch[0] = c[0] / denominator;
    x[0] = f[0] / denominator;

    for (int i = 1; i < n; i++) { /* eliminazione */
        denominator = b[i] - a[i] * scratch[i - 1];
        scratch[i] = c[i] / denominator;
        x[i] = (f[i] - a[i] * x[i - 1]) / denominator;
    }

    for (int i = n - 2; i >= 0; i--) { /* sostituzione */
        x[i] -= scratch[i] * x[i + 1];
    }
}
```

Si vede bene la catena: nel primo ciclo il valore in `i` dipende da quello in `i-1`. Non c’è modo di calcolarli in parallelo.

Il complemento di Schur rompe la catena così: dentro il mio blocco, la soluzione è la somma di due contributi. Il primo è la risposta al carico che ho io, e lo calcolo subito da solo. Il secondo è la risposta ai valori alle mie due estremità, che ancora non conosco – ma posso calcolare quanto “pesa” ciascuna estremità sul mio interno, con due sole risoluzioni locali in più.

```
/* ---- 1. ogni blocco per conto suo: tre risoluzioni locali ---- */
thomas_solve(len, a+begin, b+begin, c+begin, f+begin, y+begin, scratch);

if (p > 0) { /* influenza dell'estremità sinistra */
    memset(rhs, 0, len * sizeof(Real));
    rhs[0] = a[begin];
    thomas_solve(len, a+begin, b+begin, c+begin, rhs, lft+begin, scratch);
}

if (has_right) { /* influenza dell'estremità destra */
    memset(rhs, 0, len * sizeof(Real));
    rhs[len - 1] = c[internal_end - 1];
    thomas_solve(len, a+begin, b+begin, c+begin, rhs, rgt+begin, scratch);
}
```

y è la risposta al carico proprio; lft e rgt sono le due funzioni di influenza, ottenute risolvendo lo stesso sistema con un termine noto che vale zero ovunque tranne che a un'estremità. Nessuna di queste tre risoluzioni guarda i dati di un altro blocco.

A quel punto le uniche incognite rimaste sono i punti di giunzione fra blocchi. Sostituendo l'espressione della soluzione interna nelle equazioni scritte in quei punti si ottiene un sistema nelle sole giunzioni, che risulta essere di nuovo tridiagonale:

```
/* ---- 2. il sistema rimasto sulle giunzioni ---- */
int m = end - 1; /* il punto di giunzione */
int left = m - 1; /* ultimo punto interno del blocco p */
int right = next_begin; /* primo punto interno del blocco p+1 */

ra[p] = -a[m] * lft[left];
rb[p] = b[m] - a[m] * rgt[left] - c[m] * lft[right];
rc[p] = -c[m] * rgt[right];
rf[p] = f[m] - a[m] * y[left] - c[m] * y[right];

thomas_solve(interfaces, ra, rb, rc, rf, rx, scratch);
```

Con quattro blocchi questo sistema ha tre incognite, contro le centoventotto di partenza: è minuscolo. Risolto quello, ogni blocco ricompone la sua parte:

```
/* ---- 3. ogni blocco di nuovo per conto suo ---- */
Real x_left = (p > 0) ? rx[p - 1] : (Real)0;
Real x_right = has_right ? rx[p] : (Real)0;

for (int i = begin; i < internal_end; i++) {
    x[i] = y[i] - x_left * lft[i] - x_right * rgt[i];
}
```

Questa riga è la sovrapposizione degli effetti scritta in codice: la risposta al mio carico, meno il contributo di ciascuna estremità pesato per il valore che quell'estremità ha effettivamente assunto.

Il punto importante: **non è un'approssimazione**. È lo stesso identico risultato del metodo sequenziale, con le operazioni fatte in un ordine diverso.

In questo passo i “blocchi” sono ancora fette di un unico array dentro un unico processo, e non si comunica niente. Serve a verificare la matematica quando è ancora ispezionabile con un debugger ordinario.

B2 — Dare a ciascun processore la sua fetta

Prima: I processori esistevano ma nessuno diceva loro quale parte della griglia gli spettasse.

Dopo: I processori sono disposti su una griglia tridimensionale, e ognuno ne ricava la propria fetta.

```
void par_topology_init(const int procs[3]) {
    int dims[3] = {procs[0], procs[1], procs[2]};
    int periods[3] = {0, 0, 0};

    MPI_Dims_create(par_size(), 3, dims);
    MPI_Cart_create(MPI_COMM_WORLD, 3, dims, periods, 0, &cart_comm);
    MPI_Cart_get(cart_comm, 3, cart_dims, periods, cart_coords);

    for (int c = 0; c < 3; c++) { /* un gruppo per direzione */
        int keep[3] = {0, 0, 0};
        keep[c] = 1;
        MPI_Cart_sub(cart_comm, keep, &line_comm[c]);
    }
}
```

`MPI_Dims_create` riempie solo le direzioni lasciate a zero: passando $\{1, 1, 0\}$ si ottiene una divisione a fette lungo Z , passando $\{0, 0, 0\}$ è MPI a scegliere una forma equilibrata (con otto processori, $2 \times 2 \times 2$).

L'ultimo ciclo crea tre gruppi di processori, uno per direzione: sono i gruppi che si scambieranno le giunzioni nel complemento di Schur.

```
void decomp_init_mpi(Decomp *d) {
    int dims[3], coords[3];
    par_dims(dims);
    par_coords(coords);

    for (int c = 0; c < 3; c++) {
        int begin, end;
        decomp_share(d->n_global[c], dims[c], coords[c], &begin, &end);

        d->n[c] = end - begin;
        d->start[c] = begin;
        d->is_first[c] = (coords[c] == 0);
        d->is_last[c] = (coords[c] == dims[c] - 1);
    }
    decomp_set_layout(d, 0);
}
```

Qui si vede il senso della fase A: è *l'unica* funzione che cambia rispetto alla versione seriale. `n` diventa più piccolo, `start` non è più zero, e i flag delle pareti diventano veri solo per i processori che stanno davvero agli estremi. Tutto il resto del solutore non se ne accorge.

Con 24 celle lungo Z e cinque processori i blocchi escono di dimensione 5, 5, 5, 5, 4: il resto della divisione viene distribuito una cella per volta ai primi blocchi, così le dimensioni differiscono al massimo di uno.

B3 — Il complemento di Schur su processori veri

Prima: Il complemento di Schur funzionava su fette di un array, in un solo processo.

Dopo: Funziona con un blocco per processore, e i valori che prima si leggevano dall'array del vicino ora viaggiano come messaggi.

Le fasi 1 e 3 restano identiche a prima e non comunicano. Cambia solo la fase 2:

```

/* Il mio vicino di sinistra ha bisogno di sapere quanto vale
   il mio primo punto interno nelle tre risposte locali. */
Real mine[3] = {y[0], lft[0], rgt[0]};
Real from_right[3] = {0, 0, 0};
par_shift_real(axis, -1, mine, from_right, 3);

/* La riga del sistema sulle giunzioni che spetta a me.
   L'ultimo processore non ha giunzioni e manda una riga innocua. */
Real row[4] = {0, 1, 0, 0};

if (has_right) {
    int m = n_local - 1;
    int left = len - 1;
    row[0] = -a[m] * lft[left];
    row[1] = b[m] - a[m] * rgt[left] - c[m] * from_right[1];
    row[2] = -c[m] * from_right[2];
    row[3] = f[m] - a[m] * y[left] - c[m] * from_right[0];
}

/* Tutti ricevono tutte le righe, e tutti risolvono il sistemino:
   cosi' nessuno deve rispedire indietro la soluzione. */
par_line_allgather(axis, row, 4, rows);

```

Confrontando con il codice di B1 si vede che le formule sono le stesse: dove prima c'era `lft[right]` – un valore letto dall'array del blocco vicino – ora c'è `from_right[1]`, cioè lo stesso valore arrivato per posta.

Lo scambio col vicino è implementato così:

```

void par_shift_real(int axis, int step,
                    const Real *send, Real *recv, int count) {
    MPI_Sendrecv(send, count, PAR_REAL, mpi_neighbor(axis, step), 0,
                  recv, count, PAR_REAL, mpi_neighbor(axis, -step), 0,
                  cart_comm, MPI_STATUS_IGNORE);
}

```

Due dettagli meritano una nota.

`MPI_Sendrecv` invece di una `Send` seguita da una `Recv`: due processori affacciati devono scambiarsi dati nello stesso istante, e con chiamate separate si bloccherebbero a vicenda non appena i messaggi superano la soglia oltre la quale MPI smette di conservarli in un'area temporanea.

`mpi_neighbor` restituisce un valore speciale, `MPI_PROC_NULL`, dove il vicino non c'è. MPI tratta un invio o una ricezione verso il vuoto come un'operazione che non fa niente, quindi non serve nessun caso particolare per i processori sul bordo.

Il conto totale della comunicazione è sorprendentemente piccolo:

Fase	Comunicazione
1. Calcolo locale (tre risoluzioni)	nessuna
2. Sistema sulle giunzioni	3 numeri col vicino, 4 per processore
3. Ricomposizione locale	nessuna

Tutto il resto è calcolo che ciascun processore fa per conto proprio: è esattamente ciò che serve perché il parallelo convenga.

5. Fase C — Le celle in prestito dal vicino

Prima: Ogni processo possedeva solo le proprie celle. Ma per calcolare vicino al bordo del proprio blocco servono i dati di quello accanto.

Dopo: Ogni processo alloca un anello di celle in più tutt'intorno, dove tiene una copia dell'ultima fila del vicino, e se la fa aggiornare con un messaggio prima di usarla.

Il conto che il solutore fa in ogni cella guarda anche la cella prima e quella dopo: è una derivata seconda. Per una cella sul bordo del blocco, “quella dopo” appartiene al vicino. Le celle in prestito servono a questo, e si chiamano *celle fantasma*.

Allocarle è costato una riga

```
| decomp_set_layout(d, 1); /* prima era 0: nessun margine */
```

Questo è il ritorno dell'investimento della fase A. Il margine cambia dove stanno i dati in memoria – le righe non sono più contigue, la prima cella non è più all'indice zero – ma tutte le centinaia di calcoli di posizione sparsi nel codice passano da `decomp_index`, che legge quei numeri dalla struttura. Non è stato necessario toccarne nemmeno uno.

Lo scambio

```
for (int axis = 0; axis < 3; axis++) {
    int below = par_neighbor(axis, -1);
    int above = par_neighbor(axis, +1);

    if (below == PAR_NO_NEIGHBOR && above == PAR_NO_NEIGHBOR) {
        continue; /* direzione non divisa: niente da scambiare */
    }

    /* La mia ultima faccia va a chi sta sopra; dal basso arriva la sua. */
    face_copy(d, field, axis, d->n[axis] - 1, packet, 1);
    par_shift_real(axis, +1, packet, arrived, face);
    if (below != PAR_NO_NEIGHBOR) {
        face_copy(d, field, axis, -1, arrived, 0);
    }

    /* E specularmente verso il basso. */
    face_copy(d, field, axis, 0, packet, 1);
    par_shift_real(axis, -1, packet, arrived, face);
    if (above != PAR_NO_NEIGHBOR) {
        face_copy(d, field, axis, d->n[axis], arrived, 0);
    }
}
```

`face_copy` copia una faccia del blocco dentro un pacchetto contiguo, o viceversa: impacchetta e scarica. `par_shift_real` è la stessa funzione già usata nella fase B, quindi non c'è nessuna primitiva MPI nuova.

Due dettagli che vale la pena spiegare.

MPI mette a disposizione i *tipi derivati* proprio per descrivere dati non contigui senza copiarli a mano. Non li ho usati perché, come dice la documentazione stessa, in mancanza di hardware apposito la libreria fa comunque questa identica copia in un'area contigua. Farla a mano costa uguale e si legge; in più vale subito per tutte e tre le direzioni.

Non servono gli *angoli*. I conti guardano una cella più in là in *una* direzione alla volta, mai in diagonale: bastano le sei facce, non serve un secondo giro per gli spigoli.

Quanto si scambia

Su una griglia 128^3 divisa fra quattro processi, la faccia da spedire è 128 KB contro i 4 MB del blocco locale: si scambia circa il **3%** dei dati per poter calcolare il restante 97% in autonomia. È il rapporto fra superficie e volume, ed è la ragione per cui dividere il dominio funziona.

6. Fase D — Il primo calcolo davvero parallelo

Prima: Tutti i pezzi esistevano, ma nessuno li aveva collegati: il solutore girava ancora come se fosse solo.

Dopo: I sistemi lungo Z passano dal complemento di Schur, le celle di contorno vengono aggiornate al momento giusto, e il risultato non dipende più da quanti processi si usano.

Scrivere il sistema invece di risolverlo di corsa

Prima il codice costruiva e risolveva il sistema tridiagonale in un unico ciclo, senza mai scrivere da nessuna parte le tre diagonali. Per consegnarlo al complemento di Schur bisogna invece scriverle:

```
if (along == 0) {
    /* Parete inferiore del dominio: il valore e' imposto. */
    lower[at] = 0.0;
    diagonal[at] = 1.0;
    upper[at] = 0.0;
    known[at] = bc_left(data->bc_velocity, gi, gj, gk, t_step, v_comp);
} else if (along < last_global) {
    lower[at] = w_i;
    diagonal[at] = 1.0 - 2.0 * w_i; /* riga interna */
    upper[at] = w_i;
    known[at] = rhs;
} else {
    ... /* parete superiore */
}
```

La variabile chiave è `along`: è l'indice **globale**, cioè la posizione della cella nel dominio intero. Un processo in mezzo non ha nessuna parete, quindi per lui `along` non vale mai zero e usa ovunque la riga interna.

Questo è il punto in cui il quarto tipo di modifica della fase A – quello delle “pareti fantasma” – viene finalmente messo alla prova. Se il test fosse rimasto sull'indice locale, ogni processo avrebbe creduto di avere una parete al proprio bordo.

Il momento in cui aggiornare le celle di contorno

```
for (int t_step = 1; t_step <= STEPS; t_step++) {
    refresh_vector_halo(decomp, &solver_mem_state->eta);
    refresh_vector_halo(decomp, &solver_mem_state->zeta);
    refresh_vector_halo(decomp, &solver_mem_state->u);
    par_exchange_halo(decomp, solver_mem_state->pressure_star.v);

    momentum_step(...);

    /* La velocita' e' appena cambiata e il passo di pressione la rilegge. */
    refresh_vector_halo(decomp, &solver_mem_state->u);
    pressure_step(...);
}
```

Due soli momenti per passo temporale: all'inizio, e in mezzo dopo che la velocità è stata aggiornata. La regola è semplice: si aggiorna un campo quando è cambiato e sta per essere riletto oltre il proprio bordo.

Una comunicazione per gruppo di linee, non per linea

Questa è la scelta di progetto più importante di tutta la fase.

Il solutore chiama il complemento di Schur *una volta per ogni linea della griglia*. Su 128^3 sono circa sedicimila linee. Un messaggio per linea significherebbe sedicimila messaggi per ogni

risoluzione: costerebbe molto più del calcolo che permette di fare.

La soluzione è fare prima tutto il lavoro locale di un intero gruppo di linee, e poi scambiare una volta sola per l'intero gruppo:

```
/* 1. ogni linea per conto suo, nessuna comunicazione */
for (int l = 0; l < lines; l++) {
    thomas_solve(len, a + off, b + off, c + off, f + off, y + off, scratch);
    ... /* piu' le due funzioni di influenza */
}

/* 2. una sola volta, per tutte le linee insieme */
par_shift_real(axis, -1, mine, from_right, 3 * lines);
par_line_allgather(axis, row, 4 * lines, rows);

/* 3. ogni linea di nuovo per conto suo */
```

Si passa da migliaia di messaggi minuscoli a poche centinaia per passo temporale.

Il controllo che chiude la fase

Il complemento di Schur è un metodo *esatto*: non approssima niente, riordina soltanto le stesse operazioni. Quindi il numero di processi non deve cambiare il risultato. Il confronto con la versione a un processo solo:

	un processo	da 1 a 8 processi
errore su u_x	5.8196668537e-04	identico, cifra per cifra
errore su u_y	5.4026976672e-04	identico, cifra per cifra
errore su u_z	2.8691858457e-04	identico, cifra per cifra
errore su p	1.9758785661e-01	identico, cifra per cifra

Non era scontato ottenere un'identità *esatta*: cambiando l'ordine delle somme in virgola mobile ci si aspetterebbero scarti nelle ultime cifre. Il fatto che non ce ne siano conferma che il metodo distribuito fa anche le stesse operazioni nello stesso ordine.

Una cosa anticipata dalla fase F

Al primo tentativo gli errori sembravano *migliorare* aumentando i processi. Non era vero: ogni processo calcolava la norma dell'errore solo sul proprio blocco, senza sommare quella degli altri. È bastato aggiungere tre somme globali, ma senza di esse la fase D non aveva un modo per verificarsi.

7. Fase E — Dividere anche nelle altre due direzioni

Prima: La griglia si divideva solo lungo Z: le altre due direzioni davano risultati sbagliati, e il programma si fermava con un messaggio se qualcuno ci provava.

Dopo: Si divide in tutte e tre le direzioni, e la forma della griglia di processi la sceglie MPI.

La scoperta che ha guidato il lavoro

Servivano le versioni con Schur anche per X e Y. Ma i tre passi sono la **stessa operazione ripetuta su assi diversi** — ed è esattamente come li presenta il corso:

$$(I - \gamma \partial_{xx}) \Delta \eta = \xi - \eta^n, \quad (I - \gamma \partial_{yy}) \Delta \zeta = \eta^{n+1} - \zeta^n, \quad (I - \gamma \partial_{zz}) \Delta u = \zeta^{n+1} - u^n$$

Scrivene altre due copie avrebbe voluto dire tre funzioni quasi identiche. Ne ho scritta **una sola**, che prende l'asse come parametro:

```

/* Le linee corrono lungo 'axis'; delle altre due direzioni, 'group'
 * raccoglie le linee risolte insieme, 'outer' e' il ciclo esterno. */
const int group = (axis == 0) ? 1 : 0;
const int outer = (axis == 2) ? 1 : 2;
const int length = d->n[axis];
const int lines = d->n[group];
const size_t step = d->stride[axis];

```

`step` è la distanza in memoria fra due celle vicine lungo quell'asse: 1 lungo X, la larghezza di una riga lungo Y, quella di un piano lungo Z. Prendendola dalla struttura, lo stesso ciclo percorre linee in qualunque direzione.

Lo stesso per la pressione

Anche i tre passi della pressione hanno la *stessa matrice*: cambiano solo il campo da cui si legge e quello in cui si scrive. Una funzione sola, e la cascata del corso diventa leggibile in cinque righe:

```

compute_div(decomp, buffer, &solver_mem_state->u);
pressure_direction(decomp, 0, buffer, star); /* psi */
pressure_direction(decomp, 1, star, buffer); /* phi */
pressure_direction(decomp, 2, buffer, star); /* fi */
update_pressure(decomp, solver_mem_state);

```

I due campi si scambiano il ruolo a ogni passo, quindi ne bastano due.

Il risultato

La fase E ha **tolto** codice invece di aggiungerne:

	Prima	Dopo
Quantità di moto	3 funzioni, 297 righe	1 funzione, 150 righe
Pressione	4 funzioni, 250 righe	1 funzione, 140 righe

E non è servito toccare nulla in `parallel.c`: lo scambio delle celle di contorno era già scritto per tutti e tre gli assi fin dalla fase C, e il complemento di Schur prende l'asse come parametro fin dalla fase B. Era il motivo di quelle scelte.

8. Fase F — Scrivere i risultati in parallelo

Prima: Ogni processo avrebbe scritto il proprio pezzo di dominio in un file con lo stesso nome: si sarebbero sovrascritti a vicenda.

Dopo: Ogni processo scrive il proprio file, e il primo scrive un indice che tiene insieme i pezzi.

L'indice è un file di poche righe che elenca dove sta ciascun pezzo. Aprendo quello in ParaView si vede il dominio intero ricomposto:

```

<PIImageData WholeExtent="0 23 0 15 0 11" GhostLevel="0" ...>
  <Piece Extent="0 7 0 7 0 11" Source="sol_0005_p000.vti"/>
  <Piece Extent="0 7 8 15 0 11" Source="sol_0005_p001.vti"/>
  <Piece Extent="8 15 0 7 0 11" Source="sol_0005_p002.vti"/>
  ...

```

Nessuno raccoglie i dati

Non c'è nessuna raccolta su un processo solo: sarebbe il collo di bottiglia classico, e su griglie grandi significherebbe far transitare gigabyte attraverso un unico processo. Ognuno scrive per conto proprio, in parallelo.

E il processo che scrive l'indice **non chiede a nessuno dove stanno gli altri**:

```
for (int rank = 0; rank < par_size(); rank++) {
    int coords[3], begin[3], end[3];

    par_coords_of(rank, coords);
    for (int c = 0; c < 3; c++) {
        decomp_share(d->n_global[c], dims[c], coords[c], &begin[c], &end[c]);
    }
    fprintf(fp, " <Piece Extent=\"%d %d %d %d %d %d\" Source=\"%s\"/>\n", ...);
}
```

Conoscendo la forma della griglia di processi, ricalcola i blocchi altrui con `decomp_share`: **la stessa funzione con cui ciascuno si è preso il suo**. Una sola regola usata in due posti, quindi l'indice e i pezzi non possono contraddirsi.

9. Fase G — Quanto conviene

Le misure sono state fatte su un portatile con processore AMD Ryzen 7 7435HS: **8 nuclei fisici** (16 con il multithreading) e 7 GB di memoria. Ogni configurazione è stata ripetuta tre volte tenendo il tempo migliore: senza ripetizioni il rumore di fondo della macchina rendeva i numeri inaffidabili.

Le due domande

Lo *strong scaling* chiede: a parità di problema, quanto si accorcia il tempo aggiungendo processori? Il *weak scaling* chiede: a parità di lavoro per processore, il tempo resta lo stesso mentre il problema cresce?

proc.	Strong (problema fisso 128 ³)			Weak (64 ³ per processo)		
	tempo/pass	speedup	effic.	tempo/pass	rapporto	effic.
1	941 ms	1.00	100%	123 ms	1.00	100%
2	551 ms	1.71	85%	149 ms	0.83	83%
4	340 ms	2.77	69%	166 ms	0.74	74%
8	228 ms	4.13	52%	232 ms	0.53	53%

Da dove viene la perdita: tre misure, non tre ipotesi

Invece di attribuire la perdita a un generico “sovraccarico”, l’ho scomposta.

Contesa di memoria. Ho lanciato **otto copie indipendenti** dello stesso problema, senza *nessuna* comunicazione fra loro. Rallentamento: **1,35 volte**. È il limite della macchina, non del programma: otto nuclei che si contendono un unico canale verso la memoria. Per un codice come questo, che sposta molti più dati di quanti conti faccia, è il tetto vero.

Il tempo dentro MPI. Misurato direttamente, con un cronometro attorno alle chiamate: al massimo carico sono 37 ms su 232, cioè circa il **16%**.

La vettorizzazione che si perde. Questa è la scoperta più interessante. Confrontando i singoli stadi a parità di lavoro per processo:

stadio	1 processo	8 processi	
passo lungo X	61,0 ms	101,9 ms	×1,7
passo lungo Y	3,7 ms	33,0 ms	×8,9
passo lungo Z	3,9 ms	37,9 ms	×9,6

Quasi dieci volte, e non è colpa né di MPI né della memoria. Il progetto conteneva già dei nuclei di calcolo *vettorizzati*, che risolvono una linea intera in un colpo solo e sono molto veloci. Ma funzionano solo su una linea intera: appena una direzione viene divisa fra processi, quella scorciatoia non è più utilizzabile e si ricade sulla versione ordinaria.

Con otto processi disposti $2 \times 2 \times 2$ tutte e tre le direzioni sono divise, quindi la vettorizzazione non si usa mai. Rifacendo lo studio con la vettorizzazione spenta ovunque – confronto alla pari – lo speedup passa da 3,18 a **4,13**.

Il bilancio a otto processi: circa 35% contesa di memoria, 16% comunicazione, il resto il costo intrinseco del metodo di Schur, che fa tre risoluzioni locali al posto di una.

I grafici

Le curve sono in `docs/scaling/scaling.svg`, prodotte da `scripts/plot_scaling.py` a partire dai dati di `scripts/run_scaling.sh`. Tre pannelli: lo speedup a confronto con la retta ideale, l'efficienza del weak scaling, e le barre che mostrano quanta parte del tempo se ne va dentro MPI.

10. Come abbiamo verificato

Ogni passo ha il suo controllo automatico, che risponde “passato” o “fallito” invece di stampare numeri da interpretare a occhio.

Controllo	Che cosa verifica	Esito
Risultati invariati	Il solutore dà gli stessi numeri di prima della fase A	identici, cifra per cifra
Disposizione in memoria	Cambiando come i dati stanno in RAM, il risultato non cambia	zero celle diverse
Schur (locale)	Stessa risposta del metodo sequenziale, 24 configurazioni	scarto $\sim 10^{-17}$
Fette dei processori	I blocchi coprono la griglia una volta sola	zero buchi, zero sovrapposizioni
Schur (distribuito)	Stessa risposta del sequenziale, con i messaggi	scarto $\sim 10^{-17}$
Celle di contorno	L'anello contiene i dati del vicino, e resta intatto dove c'è la parete	zero celle sbagliate
Numero di processi	Stessi errori di soluzione con 1, 2, 3, 4, 6, 8 processi e 8 forme di divisione	identici, cifra per cifra
File di uscita	I pezzi elencati nell'indice coprono il dominio una volta sola	zero buchi, zero sovrapposizioni

Il settimo è quello decisivo, ed è stato ripetuto su due problemi di prova, due griglie, con e senza vettorizzazione, e con otto forme diverse di divisione fra cui i casi che dividono *solo* lungo X e *solo* lungo Y.

Abbiamo verificato anche i controlli stessi

Un controllo che passa sempre non dimostra niente. Per esserne certi abbiamo introdotto di proposito degli errori nel codice, uno alla volta, e verificato che venissero scoperti. Su **trentuno** errori introdotti, trenta sono stati scoperti; il trentunesimo era un *mutante equivalente*, cioè un programma scritto diversamente che calcola però la stessa identica cosa.

Questa verifica ha ripagato subito, perché ha rivelato un difetto *nei controlli stessi*. Il calcolo dello scarto massimo era scritto così:

```
if (difference > worst) {  
    worst = difference;  
}
```

Rompendo di proposito una comunicazione, il risultato diventava “non numerico” – il valore che si ottiene, ad esempio, dividendo per zero. Ma per quei valori *ogni confronto risulta falso*: il valore non entrava mai nel massimo, e il controllo dichiarava “passato” su un risultato completamente rotto. La correzione è invertire il confronto:

```
if (!(difference <= worst)) {  
    worst = difference;  
}
```

Sui numeri normali le due scritture sono equivalenti; su un valore non numerico la seconda risulta vera, quindi l’errore emerge.

11. Costi e limiti, dichiarati

Il codice seriale è diventato più lento del 12%. Per poter consegnare il sistema al complemento di Schur bisogna scrivere le tre diagonali in memoria, mentre prima venivano calcolate al volo. È il prezzo della parallelizzazione, ed è misurato: da 37,5 a 42,2 unità di tempo per cella-passo.

La vettorizzazione e la divisione oggi competono. Ogni direzione divisa è una direzione su cui la scorciatoia vettorizzata non si può più usare. Non è un difetto nascosto: è un lavoro non ancora fatto, e la strada è nota (vettorizzare il percorso di Schur, le cui linee sono già indipendenti fra loro).

Il complemento di Schur fa tre risoluzioni locali invece di una: la risposta al carico proprio più le due funzioni di influenza. Per la pressione, però, la matrice è costante nel tempo e uguale per tutte le linee: le due funzioni di influenza si potrebbero calcolare una volta sola all’avvio. È esattamente il “preprocessing” descritto a lezione, ed è l’ottimizzazione successiva più ovvia.

Un portatile non è la macchina giusta per uno studio di scaling. Con un solo canale verso la memoria, già a otto processi il limite è la banda e non il programma. Gli stessi script funzionano senza modifiche su un calcolatore con più nodi, dove i numeri sarebbero considerevolmente migliori.

12. Cosa si potrebbe fare dopo

Vettorizzare il percorso parallelo

È il guadagno più grande a portata di mano: recupererebbe il fattore dieci perso sui passi lungo Y e Z quando quelle direzioni sono divise.

Precalcolare le funzioni di influenza della pressione

La matrice non cambia mai: le due risoluzioni in più si farebbero una volta all’avvio invece che a ogni passo.

Sovrapporre comunicazione e calcolo

MPI permette di avviare un messaggio e continuare a calcolare mentre viaggia. Va misurato però se conviene davvero: la documentazione stessa avverte che senza una scheda di rete che se ne occupi da sola il guadagno spesso non c’è.

Misurare su un calcolatore vero

Gli script sono pronti; servirebbe solo una macchina con più nodi per vedere fin dove arriva il metodo.

Il solutore gira su più processori, la griglia si divide nelle tre direzioni, e il risultato non dipende da quanti processori si usano: la stessa soluzione, cifra per cifra, da uno a otto. Ogni pezzo è stato costruito e verificato singolarmente prima di essere collegato agli altri, e quando qualcosa non funzionava sapevamo dove non guardare.